

《漏洞利用及渗透测试基础》实验报告

姓名：王一诺 学号：2312331 班级：计科二班

实验名称：

格式化字符串漏洞

实验要求：

以第四章示例 4-7 代码，完成任意地址的数据获取，观察 Release 模式和 Debug 模式的差异，并进行总结。

实验过程：

1. 项目源码

```
#include <stdio.h>
int main(int argc, char *argv[])
{
    char str[200];
    fgets(str,200,stdin);
    printf(str);
    return 0;
}
```

2. Debug 模式

首先，在 ollydbg 中选中源代码在 debug 模式下编译生成的 exe 文件进行调试。

地址	十六进制数据	指令	注释	寄存器 (FPU)
004013CD	CC	INT3		EAX 00000000
004013CE	CC	INT3		ECX 0012FFB0
004013CF	CC	INT3		EDX 7C92E4F4 ASCII
004013D0	55	PUSH EBP	lab4.<Modu	EBX 7FFD5000
004013D1	8BEC	MOV EBP,ESP		ESP 0012FFC4
004013D3	6A FF	PUSH -1		EBP 0012FFF0
004013D5	68 58014200	PUSH OFFSET 00420158	ASCII	ESI 00129E9C
004013DA	68 B4624000	PUSH _except_handler3	ASCII U	EDI 0012CAEE
004013DF	64:A1 000000	MOV EAX,DWORD PTR FS:[0]		EIP 004013D0 lab4.
004013E5	50	PUSH EAX		C 1 ES 0023 32bit
004013E6	64:8925 0000	MOV DWORD PTR FS:[0],ESP	安装 <???	P 0 CS 001B 32bit
004013ED	83C4 F0	ADD ESP,-10		A 0 SS 0023 32bit
004013F0	53	PUSH EBX		Z 0 DS 0023 32bit
004013F1	56	PUSH ESI		S 1 FS 003B 32bit
004013F2	57	PUSH EDI		T 0 GS 0000 NULL
004013F3	8965 E8	MOV DWORD PTR SS:[LOCAL.6],ESP		D 0
004013F6	FF15 40514200	CALL NEAR DWORD PTR DS:[<KERNEL32.GetU	[KERNEL32.	0 0 LastErr 00000
004013FC	A3 EC354200	MOV DWORD PTR DS:[_osver],EAX		
00401401	A1 FC354200	MOV FAX.DWORD PTR DS:[_osver]	ASCII	

地址	十六进制数据	地址	数值	ASCII	注释
00422000	00 00 00 00 00 00 00 00 00 00 00 00 00 00	0012FFC4	7C817067	gp	返回到 kernel32.7C8
00422010	00 00 00 00 00 00 00 00 00 00 00 00 00 00	0012FFC8	0012CAEE		
00422020	00 00 00 00 00 00 00 00 00 00 00 00 00 00	0012FFCC	00129E9C		

向下查找，可以找到 call 00401005 指令，在这里发生 main 函数的调用。

地址	十六进制数据	指令	注释
0040148A	E8 C1430000	CALL _setenvp	[_setenvp
0040148F	E8 CC3D0000	CALL _cinit	[_cinit
00401494	8B00 0B364200	MOV ECX,DWORD PTR DS:[_environ]	ASCII
0040149A	8900 0C364200	MOV DWORD PTR DS:[_initenv],ECX	
004014A0	8B15 0B364200	MOV EDX,DWORD PTR DS:[_environ]	ASCII
004014A6	52	PUSH EDX	
004014A7	A1 0B364200	MOV EAX,DWORD PTR DS:[_argv]	ASCII
004014AC	50	PUSH EAX	
004014AD	8B00 FC354200	MOV ECX,DWORD PTR DS:[_argc]	ASCII
004014B3	51	PUSH ECX	
004014B4	E8 4CFBFFFF	CALL 00401005	[main
004014B9	83C4 0C	ADD ESP,0C	
004014BC	8945 E4	MOV DWORD PTR SS:[LOCAL.7],EAX	
004014BF	8B55 E4	MOV EDX,DWORD PTR SS:[LOCAL.7]	
004014C2	52	PUSH EDX	
004014C3	E8 D83D0000	CALL _exit	[_exit
004014C8	8B45 EC	MOV EAX,DWORD PTR SS:[LOCAL.5]	
004014CB	8B08	MOV ECX,DWORD PTR DS:[EAX]	
004014CD	AR11	MOV EDX.DWORD PTR DS:[ECX]	

按 F7 进入 main 函数的调试，如下图所示。先是 push EBP 将 ebp 寄存器压入栈中，然后 sub esp, 108，这个 108 是很大的值，远比里面的这个 200 (C8) 要大，也就是我们抬高了栈，给出了很大一片区域。接下来有三个 push，将三个寄存器压入，它们是在整个区域的顶部，保存了调用这个函数之前的那个函数的一些寄存器的值。

地址	十六进制数据	指令	注释	寄存器 (FPU)
00401004	CC	INT3		EAX 00430AD0 ASCII
00401005	E9 06000000	JMP main		ECX 00000001
0040100A	CC	INT3		EDX 00430A30 ASCII
0040100B	CC	INT3		EBX 7FFDF000
0040100C	CC	INT3		ESP 0012FF84 UNICO
0040100D	CC	INT3		EBP 0012FFC0
0040100E	CC	INT3		ESI 00129E9C
0040100F	CC	INT3		EDI 0012CAEE
00401010	55	PUSH EBP	lab4.main	EIP 00401010 lab4.
00401011	8BEC	MOV EBP,ESP		C 0 ES 0023 32Bit
00401013	81EC 00010000	SUB ESP,108		P 0 CS 001B 32Bit
00401019	53	PUSH EBX		A 0 SS 0023 32Bit
0040101A	56	PUSH ESI		Z 0 DS 0023 32Bit
0040101B	57	PUSH EDI		S 0 FS 003B 32Bit
0040101C	8DBD F8FEFF	LEA EDI,[LOCAL.66]		T 0 GS 0000 NULL
00401022	B9 42000000	MOV ECX,42		D 0
00401027	B8 CCCCCCCC	MOV EAX,CCCCCCCC		0 0 LastErr 00000
0040102C	F3AB	REP STOS DWORD PTR ES:[EDI]		
0040102F	6A 30204200	PUSH OFFSET _iob		

观察下图，可见此时 EDI、ESI、EBX 均已压入栈顶。

指令	注释	寄存器 (FPU)
INT3		EAX 00430AD0 ASCII
INT3		ECX 00000001
PUSH EBP	lab4.main	EDX 00430A30 ASCII
MOV EBP,ESP		EBX 7FFDF000
SUB ESP,108		ESP 0012FE6C
PUSH EBX		EBP 0012FF80
PUSH ESI		ESI 00129E9C
PUSH EDI		EDI 0012CAEE
LEA EDI,[LOCAL.66]		EIP 0040101C lab4.
MOV ECX,42		C 0 ES 0023 32Bit
MOV EAX,CCCCCCCC		P 1 CS 001B 32Bit
REP STOS DWORD PTR ES:[EDI]		A 1 SS 0023 32Bit
PUSH OFFSET _iob	ASCII 0?B	Z 0 DS 0023 32Bit
PUSH 0C8		S 0 FS 003B 32Bit
LEA EAX,[LOCAL.50]		T 0 GS 0000 NULL
PUSH EAX		D 0
CALL fgets	[fgets]	0 0 LastErr 00000
ADD ESP,0C		
LEA ECX,[LOCAL.50]		

地址	数值	ASCII	注释
00000000	00000000		
00000001	00000000		
00000002	00000000		
00000003	00000000		
00000004	00000000		
00000005	00000000		
00000006	00000000		
00000007	00000000		
00000008	00000000		
00000009	00000000		
0000000A	00000000		
0000000B	00000000		
0000000C	00000000		
0000000D	00000000		
0000000E	00000000		
0000000F	00000000		
00000010	00000000		
00000011	00000000		
00000012	00000000		
00000013	00000000		
00000014	00000000		
00000015	00000000		
00000016	00000000		
00000017	00000000		
00000018	00000000		
00000019	00000000		
0000001A	00000000		
0000001B	00000000		
0000001C	00000000		
0000001D	00000000		
0000001E	00000000		
0000001F	00000000		
00000020	00000000		
00000021	00000000		
00000022	00000000		
00000023	00000000		
00000024	00000000		
00000025	00000000		
00000026	00000000		
00000027	00000000		
00000028	00000000		
00000029	00000000		
0000002A	00000000		
0000002B	00000000		
0000002C	00000000		
0000002D	00000000		
0000002E	00000000		
0000002F	00000000		
00000030	00000000		
00000031	00000000		
00000032	00000000		
00000033	00000000		
00000034	00000000		
00000035	00000000		
00000036	00000000		
00000037	00000000		
00000038	00000000		
00000039	00000000		
0000003A	00000000		
0000003B	00000000		
0000003C	00000000		
0000003D	00000000		
0000003E	00000000		
0000003F	00000000		
00000040	00000000		
00000041	00000000		
00000042	00000000		
00000043	00000000		
00000044	00000000		
00000045	00000000		
00000046	00000000		
00000047	00000000		
00000048	00000000		
00000049	00000000		
0000004A	00000000		
0000004B	00000000		
0000004C	00000000		
0000004D	00000000		
0000004E	00000000		
0000004F	00000000		
00000050	00000000		
00000051	00000000		
00000052	00000000		
00000053	00000000		
00000054	00000000		
00000055	00000000		
00000056	00000000		
00000057	00000000		
00000058	00000000		
00000059	00000000		
0000005A	00000000		
0000005B	00000000		
0000005C	00000000		
0000005D	00000000		
0000005E	00000000		
0000005F	00000000		
00000060	00000000		
00000061	00000000		
00000062	00000000		
00000063	00000000		
00000064	00000000		
00000065	00000000		
00000066	00000000		
00000067	00000000		
00000068	00000000		
00000069	00000000		
0000006A	00000000		
0000006B	00000000		
0000006C	00000000		
0000006D	00000000		
0000006E	00000000		
0000006F	00000000		
00000070	00000000		
00000071	00000000		
00000072	00000000		
00000073	00000000		
00000074	00000000		
00000075	00000000		
00000076	00000000		
00000077	00000000		
00000078	00000000		
00000079	00000000		
0000007A	00000000		
0000007B	00000000		
0000007C	00000000		
0000007D	00000000		
0000007E	00000000		
0000007F	00000000		
00000080	00000000		
00000081	00000000		
00000082	00000000		
00000083	00000000		
00000084	00000000		
00000085	00000000		
00000086	00000000		
00000087	00000000		
00000088	00000000		
00000089	00000000		
0000008A	00000000		
0000008B	00000000		
0000008C	00000000		
0000008D	00000000		
0000008E	00000000		
0000008F	00000000		
00000090	00000000		
00000091	00000000		
00000092	00000000		
00000093	00000000		
00000094	00000000		
00000095	00000000		
00000096	00000000		
00000097	00000000		
00000098	00000000		
00000099	00000000		
0000009A	00000000		
0000009B	00000000		
0000009C	00000000		
0000009D	00000000		
0000009E	00000000		
0000009F	00000000		
000000A0	00000000		
000000A1	00000000		
000000A2	00000000		
000000A3	00000000		
000000A4	00000000		
000000A5	00000000		
000000A6	00000000		
000000A7	00000000		
000000A8	00000000		
000000A9	00000000		
000000AA	00000000		
000000AB	00000000		
000000AC	00000000		
000000AD	00000000		
000000AE	00000000		
000000AF	00000000		
000000B0	00000000		
000000B1	00000000		
000000B2	00000000		
000000B3	00000000		
000000B4	00000000		
000000B5	00000000		
000000B6	00000000		
000000B7	00000000		
000000B8	00000000		
000000B9	00000000		
000000BA	00000000		
000000BB	00000000		
000000BC	00000000		
000000BD	00000000		
000000BE	00000000		
000000BF	00000000		
000000C0	00000000		
000000C1	00000000		
000000C2	00000000		
000000C3	00000000		
000000C4	00000000		
000000C5	00000000		
000000C6	00000000		
000000C7	00000000		
000000C8	00000000		
000000C9	00000000		
000000CA	00000000		
000000CB	00000000		
000000CC	00000000		
000000CD	00000000		
000000CE	00000000		
000000CF	00000000		
000000D0	00000000		
000000D1	00000000		
000000D2	00000000		
000000D3	00000000		
000000D4	00000000		
000000D5	00000000		
000000D6	00000000		
000000D7	00000000		
000000D8	00000000		
000000D9	00000000		
000000DA	00000000		
000000DB	00000000		
000000DC	00000000		
000000DD	00000000		
000000DE	00000000		
000000DF	00000000		
000000E0	00000000		
000000E1	00000000		
000000E2	00000000		
000000E3	00000000		
000000E4	00000000		
000000E5	00000000		
000000E6	00000000		
000000E7	00000000		
000000E8	00000000		
000000E9	00000000		
000000EA	00000000		
000000EB	00000000		
000000EC	00000000		
000000ED	00000000		
000000EE	00000000		
000000EF	00000000		
000000F0	00000000		
000000F1	00000000		
000000F2	00000000		
000000F3	00000000		
000000F4	00000000		
000000F5	00000000		
000000F6	00000000		
000000F7	00000000		
000000F8	00000000		
000000F9	00000000		
000000FA	00000000		
000000FB	00000000		
000000FC	00000000		
000000FD	00000000		
000000FE	00000000		
000000FF	00000000		

如左下图，随后四句代码是把开辟的空间全部赋值为 CCCCCCch。可以从右下图看出，整个栈都被赋值为 CCCCCCch。

||
||
||

在测试终端输入 AAAA%X%X%X%X，观察可见其已入栈，如下图所示。

地址	数值	ASCII	注释
0012FE68	0012FEB8	ASCII "AAAA%X%X%X%X"	
0012FE6C	0012CAE6	ASCII "AAAA%X%X%X%X"	
0012FE70	00129E9C	ASCII "AAAA%X%X%X%X"	
0012FE74	7FFDE000		
0012FE78	CCCCCCCC		
0012FE7C	CCCCCCCC		
0012FE80	CCCCCCCC		

接下来我们执行完该程序，观察和分析返回结果：

地址	数值	ASCII	注释
0012FE68	0012FEB8	ASCII "AAAA%X%X%X%X"	
0012FE6C	FFFFFFFF	ASCII "AAAA%X%X%X%X"	
0012FE70	00000015		
0012FE74	7FFDF000		
0012FE78	CCCCCCCC		
0012FE7C	CCCCCCCC		
0012FE80	CCCCCCCC		

可以看出，程序正常地输出了 AAAA，但当程序遇到 %X 时，就会顺次读取和输出后面的地址。该实验中输入了四个 %X，也就是顺次向下的四个地址，即 FFFFFFFF、15、7FFDF000、CCCCCCCC 跟在 AAAA 后被输出。

3. Release 模式

在 ollydbg 中选中源代码在 debug 模式下编译生成的 exe 文件进行调试，向下查找，在三个 push 指令后面可以找到 main 函数入口：

地址	十六进制指令	注释
0040110C	> 83C AND DWORD PTR SS:[EBP-4],0000	
004011E0	. E8 CALL 00401C46	
004011E5	. FF CALL NEAR DWORD PTR DS:[<KERN	[KERNEL32.GetCommandLin
004011E8	. A3 MOV DWORD PTR DS:[406DE4],EAX	
004011F0	. E8 CALL 0040255F	[maincpp.0040255F
004011F5	. A3 MOV DWORD PTR DS:[4068C4],EAX	
004011FA	. E8 CALL 00402312	[maincpp.00402312
004011FF	. E8 CALL 00402259	[maincpp.00402259
00401204	. E8 CALL 00401EC6	[maincpp.00401EC6
00401209	. A1 MOV EAX,DWORD PTR DS:[406900]	ASCII "PMA"
0040120E	. A3 MOV DWORD PTR DS:[406904],EAX	
00401213	. 50 PUSH EAX	
00401214	. FF3 PUSH DWORD PTR DS:[4068F8]	Arg3 => [406900] = ASC
0040121A	. FF3 PUSH DWORD PTR DS:[4068F4]	Arg2 = 410010
00401220	. E8 CALL 00401000	Arg1 = 1
00401225	. 83C ADD ESP,0C	[maincpp.00401000
00401228	. 894 MOV DWORD PTR SS:[EBP-1C],EAX	
0040122B	. 50 PUSH EAX	
0040122C	. E8 CALL 00401EF3	

按 F7 进入主函数，此时已经可以看出，release 模式下的汇编代码分布明显变得紧凑，调试信息少了一些：

地址	十六进制指令	注释	寄存器 (FPU)
00401000	\$ 81C SUB ESP,0C8	maincpp.00401000(推测)	EAX 00410090 ASCII "PMA"
00401006	. 8D4 LEA EAX,[LOCAL.49]		ECX 00406000 ASCII
0040100A	. 68 PUSH OFFSET 00406030		EDX 00000003
0040100F	. 68 PUSH 0C8		EBX 7FFD9000
00401014	. 50 PUSH EAX		ESP 0012FF84
00401015	. E8 CALL 00401061		EBP 0012FFC0
0040101A	. 8D4 LEA ECX,[LOCAL.49]		ESI 00129E9C
0040101E	. 51 PUSH ECX		EDI 0012CAF8
0040101F	. E8 CALL 00401030		EIP 00401000 maincpp.004
00401024	. 33C XOR EAX,EAX		C 0 ES 0023 32Bit 0(FFF
00401026	. 81C ADD ESP,008		P 1 CS 001B 32Bit 0(FFF
0040102C	. C3 RETN		

观察发现，release 模式下并没有 EBP 入栈，而我们发现了一个 sub 抬高。我们知道主函数里面就是有一个二百字节的字符数组，那么抬高了 0C8 就是抬高了 200，仅仅是给局部变量开辟了空间，这又与 debug 模式不同。此外 debug 模式里存在的一堆 push 寄存器操作这里也没有。可见其简洁，追求运行的效率。

然后执行 print 函数，第一行 0012FEB8 为 printf 的参数即字符串的地址。后面与 debug 模式相同，先输出 AAAA，接着四个 %x 格式化操作符读取后四行的地址作为输出，因此输出结果为 AAAA12FEB8B40603041414141。如后两张图所示。



最后，将 eax 寄存器置零，将 esp 寄存器归于原位，完成了栈帧的恢复。

4. Debug vs Release

通过以上分析，可以总结出两种模式的以下几点不同：

1>Debug 模式下，栈空间分配较大，存在大量初始化数据，而 Release 模式下，栈空间分配紧凑，代码简洁，更注重运行效率。

2>Release 模式下，main 函数内没有严格的栈帧切换，也不用 push 寄存器来保存原有的值，而在程序最后有栈帧的恢复操作。

心得体会：

1. 通过本次实验，对格式化字符串漏洞有了更具体更深入的理解。
2. 在实验中对 Debug 和 Release 模式下都进行了实验和观察分析，对两种模式的异同有了直观的感受和一定的了解。
3. 对%型字符可能带来的漏洞有了直观的感受，也启发我在编写程序时，在追求正确性和高效之外，安全性也应该是我们多加考虑的方面，对栈溢出、堆溢出等等可能存在的漏洞做好检查和防范。